स्मृ

INDUSTRIAL KNOWLEDGE INTELLIGENCE PLATFORM

SMRITI

The Unified Asset & Operations Brain — a plant's institutional memory that reads its drawings, reasons across its history, proves its answers, and warns before the next failure repeats.

Working prototype
runs fully local

Tri-modal fabric
text · graph · OCR-free visual

5 agents
evidence-constrained, all cited

github.com/infinity2147/Bhadra-ET
full source + eval

Every plant already has the knowledge to prevent its next failure — it is simply trapped in a dozen disconnected systems and in the heads of retiring engineers. SMRITI (Sanskrit: **स्मृति**, "memory") fuses the entire document estate — P&IDs, work orders, SOPs, inspections, incidents, scanned OEM manuals, permits, email and real Indian regulations — into one typed knowledge graph and a tri-modal retrieval fabric you can talk to. It answers in plain language, on the drawing itself, with citations you can click through, and it converts the plant's own near-misses into warnings issued **before** the work happens.

$3.1B	318	96.6%	18
Cognite acquisition, Jun 2026 — market proven	graph nodes · 492 typed edges (reference fabric)	faithfulness (LLM-judged, 33-Q golden set)	real OISD / Factories Act / PESO clauses

Contents

1 · The problem

A refinery's operating knowledge is fragmented across **twelve disconnected systems** — the CMMS, the drawing vault, the document management system, inspection databases, incident logs, vendor manuals, permit books and email — and, critically, in the experience of **senior engineers who are retiring**. The consequences are concrete:

SILOED

Nothing is connected

The drawing that shows what feeds a pump lives nowhere near that pump's failure history or the manual that says how to fix it.

UNDIGITISED

The richest data is images

P&IDs and scanned OEM manuals are pictures. Keyword search and text RAG are blind to them.

TRIBAL

Memory walks out the door

When the reliability engineer who "has seen this three times since 2019" retires, that pattern-recognition retires too.

REACTIVE

Failures repeat

The same near-miss recurs because no one connects tomorrow's permit to the incident from three monsoons ago.

2 · Market opportunity

This is a validated, active market. In **June 2026, Schneider Electric / AVEVA acquired Cognite for $3.1 billion** — an industrial-DataOps / knowledge platform. But the incumbents are heavyweight, Western, enterprise-priced, and carry nine-to-twelve-month rollouts. SMRITI is the sharper, deployable slice, with a deliberate wedge:

- **India-first** — real OISD, Factories Act 1948 and PESO compliance built into the graph, not bolted on.
- **Reads the whole estate visually** — OCR-free, drawings and scanned manuals included.
- **Foresight, not just search** — the plant's own near-misses become warnings before the next incident.
- **Runs local** — no cloud dependency; fits an air-gapped plant network.

3 · What SMRITI is

Not another RAG chatbot. A tri-modal operations brain.

A single question fans out across three modalities — **text**, a **knowledge graph**, and the **drawings themselves** — is fused and reranked, then answered on the P&ID with inline citations. The signature question shows why one modality is not enough:

> **"P-101 keeps tripping on high temperature — what feeds it, has this happened before, and what does the manual say to check?"**
>
> This needs **drawing connectivity** (what feeds P-101) **+ work-order history** (has this happened) **+** a **visually-retrieved scanned OEM page** (what to check). A connectivity-only P&ID tool cannot produce it; neither can a text-only RAG.

01 · FUSION

Three modalities, one cited answer

02 · VISION

OCR-free visual retrieval across the corpus

03 · REASONING

The graph connects what no human does

04 · TRUST

Every sentence cites; "insufficient evidence" is valid

4 · System architecture

Six layers, all running in-process on a laptop. A dedicated one-page diagram is in architecture.html / `architecture.pdf`.

Layer	What it does
Sources	P&IDs · work orders (CMMS) · SOPs w/ revision chains · inspections · incidents/near-miss · scanned OEM manuals · permits · email · OISD/Factories Act/PESO clauses (sourced).
Ingestion (Agent 1)	Per-type parsers; LLM entity/relation extraction against the typed schema; entity resolution by canonical tag. **Continuous intake** adds structured forms, bulk CMMS CSV/JSON, and auto-classifying document upload at runtime.
Knowledge store	Industrial Knowledge Graph (NetworkX, typed ontology, provenance on every node/edge, `SUPERSEDES` version chains) + Qdrant embedded: text (dense bge-small + sparse BM25) and visual (colSmol-256M multivector, MaxSim) + a DrawingRegion index joining pixels to graph nodes.
Retrieval fabric	Intent router → text ‖ graph ‖ visual fan-out → RRF fusion → cross-encoder rerank → provenance + drawing overlays.
Agents	Expert Copilot · Diagnostics/RCA · Compliance · Lessons & Proactive Warnings — all evidence-constrained, all cited.
Presentation	Vanilla-JS PWA: streaming chat with `[c#]` citations, a Drawing Viewer with live overlays and feed-path tracing, a reasoning-trace panel, and dashboards.

5 · One query, end to end

> user asks → router classifies intent & tags → tri-modal fan-out (text ‖ graph ‖ visual) → merge + cross-encoder rerank → provenance attached (doc, page, bbox, graph nodes) → answer streams with inline `[c#]` citations → if a drawing is in evidence, the viewer renders it with regions highlighted and the feed path traced → every step is logged to the reasoning trace **and** the eval harness.

The copilot is multi-turn: a follow-up such as "what about P-103?" is resolved against the conversation via a fast-model query rewrite before retrieval, so pronouns and ellipsis work.

6 · Knowledge graph & ontology

The graph is typed and provenanced. Node types include Equipment, Area, System, Person, Document, DrawingRegion, WorkOrder, Inspection, Incident, Procedure, RegulatoryClause, FailureMode and Parameter; edges include `FEEDS_INTO`, `LOCATED_IN`, `MAINTAINED_BY`, `INSPECTED_BY`, `GOVERNED_BY`, `SIMILAR_TO`, `HAS_FAILURE_MODE`, `SUPERSEDES`, `INVOLVES` and `AUTHORED_BY`.

A record and its citable document are **separate nodes**: a work order produces both a `Document` node (the citation target) and a `WorkOrder` record node with a `:rec` suffix holding the structured fields. Every node and edge carries a `Provenance` object (`source_doc_id, page, bbox, extractor, confidence, effective_date`).

7 · Tri-modal retrieval fabric

TEXT

Hybrid dense + sparse

bge-small dense vectors + BM25 sparse, fused with Reciprocal Rank Fusion in Qdrant — semantics and exact-keyword both matter in a plant.

GRAPH

Typed traversal

Neighborhood walks over typed edges surface connectivity and history no flat index holds — e.g. the fleet-wide seal pattern.

VISUAL

OCR-free late interaction

Every page is embedded as an image (colSmol-256M, patch-level MaxSim). Scanned, stamped, skewed pages — even a 1958 legacy drawing — are retrieved on visual signal alone.

FUSION

Rerank + provenance

Candidates merge and pass a cross-encoder reranker; provenance and drawing overlays are attached before answering.

8 · The five agents

#	Agent	What it does
1	**Ingestion / KG**	Builds and grows the fabric — batch at install and incrementally at runtime.
2	**Expert Copilot**	Streaming, multi-turn, evidence-constrained Q&A with inline citations and drawing overlays.
3	**Diagnostics / RCA**	Graph-native root-cause: failure timeline, evidence-ranked causes citing record IDs, cross-asset patterns; confirmed RCAs written back as org memory.
4	**Compliance**	Maps real clauses to equipment and evaluates status against actual records; generates audit packs.
5	**Lessons & Proactive Warnings**	Clusters incidents, mines precursor signatures, and matches upcoming permits against them — warning before the work.

9 · Continuous intake — built to run for years

A customer files new records every day; a frozen fabric is not a product. SMRITI closes this gap with a first-class intake path whose **prime directive is graph parity**: a hand-entered record writes the **byte-for-byte same graph structure** (same `:rec` node ids, node types, edges and field keys) as the batch ingest — verified by a live graph-structure diff. So a new record is instantly in the timeline, analyzable by RCA, and eligible for warning-matching, with **no restart and no re-ingest**.

- **Structured forms** — typed endpoints `/api/records/{incident,work-order,inspection,permit}`, stamped `manual_intake · confidence 1.0` for audit.
- **Bulk CMMS import** — a CSV/JSON export from SAP PM / Maximo maps column→field with no per-row LLM; years of history load in one shot.
- **Auto-classifying upload** — drop any document; one model call classifies it and extracts it into the correct typed record; the unstructured long tail (email, manuals) is text- and visually-indexed.

The live foresight beat

Log an upcoming confined-space permit on TK-401 with a shortened purge, and the Lessons agent re-assembles the precursor signature of the 2019 / 2022 H_2S near-misses and fires a warning **on the spot**. The matching is computed live from the plant's own incident history.

10 · India-first regulatory intelligence

Eighteen real clauses — **OISD-STD-116/129/132/105**, **Factories Act 1948 §§31/36/37/38/40**, **SMPV(U) 2016**, **Petroleum Rules 2002** — each carrying its source URL and a verbatim flag, mapped to equipment by `GOVERNED_BY` edges and evaluated against actual records.

It catches what humans miss

SMRITI flags that the plant's own inspection plan assumes a 5-year interval where **OISD-129 cl. 11.1 requires annual**, and that **PSV-1101's relief-valve test is overdue** (OISD-132). Real gaps, each cited to the clause — and the register isn't all red, so the compliant majority is demonstrable too.

11 · The corpus — grounded, not invented

The reference plant ("Bharat Petrochem Ltd., Refinery Unit 4", a Mumbai coastal CDU support system commissioned 1998) is fictional, but **equipment physics, failure modes and regulations are real** (ISO-14224 failure taxonomy; researched OISD/Factories Act/PESO clauses with sources). Every document references the same tags, people and history, so the graph forms genuine cross-document links.

Type	Count	Notes
P&IDs	2	SVG→PNG + curated DrawingRegion ground truth (D-CW-104 r3, D-ET-401 r1)
Work orders	53	structured + rendered PDF; incl. the monsoon-seal history
Inspections	23	STR-101 ΔP series, E-201 fouling, tank & PSV reports
SOPs	12	incl. the SOP-CW-012 rev-1→3 version chain
Incidents / near-miss	10	incl. NM-2019-07, NM-2022-31 confined-space precursors
OEM manuals	3	scanned-look pages for visual retrieval (BurgFlow IOM troubleshooting)
Regulatory clauses	18	real clause text + source URL
Email	8	the retiring-engineer thread (knowledge-attribution story)
Permits	4	incl. the upcoming TK-401 confined-space permit

Planted patterns the agents must *discover*: monsoon seal failures across P-101/P-103/P-107 (shared seal model BurgFlow MS-40D); confined-space purge-shortening near-misses; compliance gaps against real clauses; an SOP version chain encoding a lesson learned; and a retiring expert whose `AUTHORED_BY` edges make the knowledge-cliff concrete.

12 · Evaluation — measured, not claimed

A custom LLM-judge harness scores claim-level faithfulness, citation correctness and expected-point coverage over a 33-question expert golden set, comparing SMRITI against a vanilla single-vector RAG baseline that shares the same answer model and corpus — so the deltas are honest.

Metric	SMRITI	Vanilla RAG baseline
Faithfulness	**96.6%**	95.6%
Citation correctness	**96.6%**	95.3%
Expected-point coverage	**86.4%**	84.2%

Aggregate deltas are modest because both systems share the answer model; the fabric's wins concentrate where it should — **multi-modal fusion +15 pts**, **proactive warnings +17 pts**, **honest partial-evidence answers +50 pts**, RCA +5 pts coverage. Two caveats we state rather than hide: a text-only judge cannot score what the drawing overlays show (under-crediting SMRITI's visual answers), and judging uses the same model family as answering.

13 · Grounded engineering decisions

Target machine: Apple-Silicon Mac, 8 GB RAM, no Docker, no API key (Claude Code subscription via the `claude` CLI). Every stack choice is tied to a verified constraint.

Area	Decision	Why
LLM access	`claude` CLI headless (JSON + streaming); SDK behind `ANTHROPIC_API_KEY`	No API key on machine; CLI verified; `--setting-sources ""` cuts 27k→6.7k token overhead. Router: haiku; answers: sonnet-tier.
Visual model	**colSmol-256M** (colpali-engine, torch 2.5.1 MPS-safe)	ColQwen3/colpali-v1.3 are 3–8B (impossible on 8 GB); colSmol is the smallest supported ColPali-family model (~1 GB), Apache/MIT. vLLM has no macOS support.
Vector store	**Qdrant embedded** (local path mode)	Docker daemon unavailable; embedded mode supports `MAX_SIM` multivector since v1.10; ~160 MB index at ~300 pages.
Text embeddings	fastembed bge-small + BM25, RRF	Best small-model quality/speed on CPU; SPLADE++ measured ~3 s/doc (impractical).
Reranker	fastembed MiniLM-L-6-v2 cross-encoder	~100–300 ms / 50 candidates; bge-reranker-base is 1 GB and ~10× slower.
Graph store	**NetworkX** MultiDiGraph, JSON-persisted, typed by our ontology layer	Neo4j = JVM + hundreds of MB; graph is <10k nodes; ontology/provenance contracts enforced in code → portable to Neo4j.
P&ID digitizer	Hero drawings authored as SVG emitting pixel-exact region + connectivity ground truth	Same data contract a CV digitizer would emit; training detectors on 8 GB/no-GPU is not credible. Honest scoping per spec §3.4c.
Eval	Custom LLM-judge harness (~150 lines) + baseline	ragas drags langchain + needs SDK auth; our judge runs over the same CLI client.
PDF/parse	pymupdf (parse + render + authoring)	8–12× faster than pdfplumber; also authors the synthetic corpus so ingestion parses real files.
Frontend	Vanilla JS + SVG PWA, no build step	Build tooling eats scarce disk; SSE chat + SVG viewer + trace panel don't need a framework.

14 · Honest scoping & production path

- **Hero P&IDs** carry authored digitizer-output ground truth; the production CV pipeline (symbol detection → tag association → line tracing) is specified as the production path. The long tail is covered by visual retrieval + page-region grounding.
- **Graph store** is NetworkX with a Neo4j-portable typed-ontology/provenance contract; swapping is a storage-layer change.
- **Scale path**: MUVERA fixed-dim encodings + int8 for the multivector index; LazyGraphRAG-style incremental indexing; RBAC on an audit-logged tool bus.
- **Next**: human-in-the-loop review queue for low-confidence extractions; a live CMMS connector; undo/versioning on hand-entered records; idempotent bulk re-import.

15 · Documentation & how to run

Documentation: product README; this overview; the architecture diagram; engineering decisions (`docs/decisions.md`); the reference dataset (`docs/corpus-design.md`); reproducible evaluation (`eval/`).

```
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
cd smriti && ../.venv/bin/python scripts/gen_corpus.py
cd backend && ../../.venv/bin/python -m smriti.ingest  # add --no-visual for a quick start
../../.venv/bin/python -m uvicorn smriti.api:app --port 8000  # open http://localhost:8000
```